

W14D04 — FastAPI + SQLAlchemy Integration

ML Engineer Journey | Phase 4 — Database & Observability | 11 Jun 2026

1. Konsep Inti — Session-per-Request Pattern

Bayangkan database sebagai perpustakaan. Setiap pengunjung (HTTP request) perlu kartu anggota sementara untuk meminjam atau mengembalikan buku. Kartu itu dibuat saat pengunjung masuk, dipakai selama transaksi, dan **dimusnahkan saat pengunjung keluar** — tidak peduli transaksinya berhasil atau gagal. Kalau kartu tidak dimusnahkan, slot kartu habis dan perpustakaan tidak bisa menerima pengunjung baru. Itulah *session leak* di SQLAlchemy.

Di FastAPI, SQLAlchemy session harus hidup hanya selama satu request cycle. Pola yang digunakan adalah **generator function** dengan **Dependency Injection**: session dibuka sebelum *yield*, di-inject ke endpoint, lalu ditutup di blok *finally* setelah response dikirim.

2. Dependency Function get_db()

```
def get_db() -> Generator[Session, None, None]:
    db = SessionLocal() # Buka session baru
    try:
        yield db # Inject ke endpoint — FastAPI pause di sini
        # Kode endpoint berjalan...
    finally:
        db.close() # SELALU tutup — bahkan kalau ada exception
```

Kenapa finally bukan except? Blok *except* hanya dieksekusi saat terjadi error. Kalau request berhasil 100%, *except* tidak pernah jalan, sehingga *db.close()* tidak dipanggil dan koneksi menggantung di pool. Blok *finally* bersifat absolut — dijalankan baik saat sukses maupun saat error, memastikan koneksi selalu dikembalikan ke pool.

3. Dependency Injection vs Tanpa DI

Aspek	Tanpa DI (naif)	Dengan DI — Depends(get_db)
Session dibuat	Di dalam endpoint langsung	Di get_db(), di-inject
Session ditutup	Harus ingat manual	Otomatis di finally
Testing	Sulit — session hardcoded	Mudah — bisa override dependency
Error handling	Session bisa leak jika exception	finally selalu jalan
Reuse	Tidak bisa	Depends(get_db) di endpoint mana saja

4. Arsitektur File

Pembagian tanggung jawab mengikuti prinsip Separation of Concerns (W11D04). Setiap file memiliki satu tanggung jawab yang jelas:

File	Tanggung Jawab
app/database.py	engine, SessionLocal, Base, get_db()
app/models.py	SQLAlchemy ORM model definition
app/schemas.py	Pydantic schema — request & response DTO
app/routers/analysis.py	FastAPI endpoint CRUD
main.py	Entry point — registrasi router, create_all

5. Pydantic Schemas — DTO Pattern

Pisahkan representasi data untuk input dan output. Client tidak boleh mengirim *id* atau *created_at* — server yang generate. Tapi response harus punya keduanya. Ini adalah pola **DTO (Data Transfer Object)**.

Schema Class	Digunakan untuk	Field kritis
AnalysisRequestCreate	Input dari client (POST body)	title, parameters, status, scenarios
AnalysisRequestUpdate	Input PATCH — partial update	title?, status? — semua Optional
AnalysisRequestResponse	Output ke client	Tambah id, created_at, updated_at
ScenarioResultCreate	Nested input di POST body	scenario_type, risk_score, dll
ScenarioResultResponse	Nested output di response	Tambah id, request_id, created_at

Wajib tambahkan `model_config = ConfigDict(from_attributes=True)` di semua Response schema agar Pydantic bisa membaca SQLAlchemy ORM object (bukan dict biasa).

6. Pola Endpoint & Hal-Hal Kritis

POST — Create dengan Child (flush sebelum loop)

Database relasional bekerja dengan hukum sebab-akibat: anak tidak bisa lahir tanpa identitas induk. Saat `db.add(db_request)` dijalankan, data masih mengambang di memori Python dan Postgres belum men-generate UUID. Jika langsung loop insert child, `db_request.id` masih *None* dan Postgres akan menolak foreign key constraint.

```
db.add(db_request)
db.flush() # Kirim INSERT ke DB → UUID ter-generate, transaksi belum selesai
for sc in payload.scenarios:
    db_scenario = ScenarioResult(request_id=db_request.id, ...) # id sudah ada
    db.add(db_scenario)
db.commit() # Kunci permanen semua perubahan dalam satu transaksi
db.refresh(db_request) # Reload dari DB agar created_at server terisi
```

PATCH — Partial Update dengan `exclude_unset=True`

model_dump() biasa mengeluarkan semua field termasuk yang tidak dikirim client — nilainya *None*. Kalau *setattr* loop jalan dengan *None*, field yang tidak dikirim akan ter-overwrite jadi *None*. **model_dump(exclude_unset=True)** hanya mengeluarkan field yang benar-benar dikirim client.

```
update_data = payload.model_dump(exclude_unset=True)
# Contoh: client hanya kirim {"status": "completed"}
# update_data = {"status": "completed"} – title tidak ikut
for key, value in update_data.items():
    setattr(db_request, key, value) # Dirty tracking SQLAlchemy
db.commit()
db.refresh(db_request) # updated_at otomatis bergeser via onupdate=func.now()
```

GET — db.get() vs select() untuk PK lookup

db.get(Model, pk) lebih efisien dari *select()* untuk Primary Key lookup karena SQLAlchemy mengecek *identity map* (cache session) terlebih dahulu sebelum query ke DB. Gunakan *db.get()* untuk PK, gunakan *select().where()* untuk kondisi lain.

7. Lazy Loading vs Eager Loading di FastAPI

Di FastAPI dengan pola *get_db()*, session ditutup di *finally* segera setelah endpoint return. Pydantic melakukan serialisasi response *setelah* endpoint return — pada saat itu session sudah *db.close()*. Jika relationship masih menggunakan lazy loading default (*lazy='select'*), SQLAlchemy akan mencoba query ke DB saat Pydantic akses *obj.scenarios*, tapi session sudah tutup → **DetachedInstanceError**.

Solusi: tambahkan **lazy='selectin'** di relationship definition. SQLAlchemy akan otomatis menembakkan query kedua (SELECT ... WHERE request_id IN (...)) saat parent dimuat, sebelum session ditutup.

```
scenarios: Mapped[List["ScenarioResult"]] = relationship(
    "ScenarioResult",
    back_populates="request",
    cascade="all, delete-orphan",
    lazy="selectin" # ← eager load otomatis, tidak perlu selectinload() manual
)
```

lazy setting	Behavior	Cocok untuk
lazy='select' (default)	Query ke DB saat attribute diakses pertama kali	Script CLI, bukan FastAPI
lazy='selectin'	2 query total: parent + semua child sekaligus	FastAPI — relationship selalu dibutuhkan
lazy='raise'	Throw error jika ada lazy load	Production ketat — force explicit eager load
selectinload() manual	Eager load hanya di query tertentu	Endpoint spesifik yang butuh nested data

8. Filter Dinamis — ilike & Chained where()

Pola: build query base dulu, tambahkan kondisi filter secara dinamis hanya jika parameter ada. Scalable — kalau ada 10 filter parameter, tinggal tambah *if* block.

```
stmt = select(AnalysisRequest)
if search:
    stmt = stmt.where(AnalysisRequest.title.ilike(f"%{search}%")) # case-insensitive
if status:
```

```
stmt = stmt.where(AnalysisRequest.status == status) # exact match
stmt = stmt.offset(skip).limit(limit)
requests = db.execute(stmt).scalars().all()
```

ilike vs like: ilike case-insensitive — 'pasar' match 'Pasar', 'PASAR', 'Simulasi Pasar'. like case-sensitive. Untuk substring search yang user-friendly, selalu gunakan ilike.

9. DatabaseManager vs get_db() — Kapan Pakai Mana

Project ini memiliki dua mekanisme session management yang sengaja dipisahkan untuk tujuan berbeda:

Mekanisme	Digunakan untuk	Behavior
get_db() generator	FastAPI endpoint (HTTP request)	Tidak auto-commit — endpoint yang commit eksplisit
DatabaseManager.session()	Script CLI, migration, job	Auto-commit jika sukses, auto-rollback jika error

Jangan campur keduanya. FastAPI endpoint harus selalu pakai Depends(get_db) agar kontrol transaksi tetap di tangan endpoint.

10. Quick Reference — Endpoint Pattern

Method	Endpoint	Status	Hal Kritis
GET	/analysis/	200	select().offset().limit() + filter dinamis
GET	/analysis/{id}	200 / 404 / 422	db.get() untuk PK, UUID validation
POST	/analysis/	201	flush() sebelum child, commit() satu transaksi
PATCH	/analysis/{id}	200 / 404 / 422	exclude_unset=True, setattr loop
DELETE	/analysis/{id}	204 / 404 / 422	db.delete() + cascade ke child otomatis